DualCodeDetect: Zero-Shot LLM-Generated Code Detection via Dual-Channel Perturbation

ZHENGDAO LI*, University of Science and Technology of China, China
XIUWEI SHANG*, University of Science and Technology of China, China
ZHENKAN FU, University of Science and Technology of China, China
SHIKAI GUO, Dalian Maritime University, China
WEIMING ZHANG, University of Science and Technology of China, China
NENGHAI YU, University of Science and Technology of China, China
KEJIANG CHEN†, University of Science and Technology of China, China

The rapid advancement of large language models (LLMs) in code generation has greatly improved software development efficiency, but it has also raised concerns about misuse, making the distinction between human-written and LLM-generated code an urgent task. However, existing detection methods for LLM-generated content, particularly perturbation-based zero-shot methods, are primarily designed for natural language scenarios and fail to transfer effectively to the task of detecting generated code. When directly applied to code, they face two major challenges: (1) the low-entropy nature of code restricts the perturbation space and weakens discriminative signals; and (2) prior perturbation methods often compromise semantic integrity or executability, leading to substantial performance degradation. To address these issues, we propose DualCodeDetect, a novel zero-shot detection framework that amplifies the differences between LLM-generated and human-written code through a dual-channel perturbation mechanism. In the semantic channel, we design an identifier perturbation strategy based on outside-nucleus sampling, which disrupts the strong consistency of LLMs in identifier selection. In the structural channel, empirical analysis reveals that LLM-generated code exhibits greater uniformity in stylistic features; leveraging this insight, we construct a rule-based library of semantics-preserving code transformations to introduce structural perturbations that further magnify statistical disparities. In experiments conducted across two datasets and ten representative code LLMs, DualCodeDetect achieves an average AUROC of 0.8477, alongside FPR and FNR values of 0.0430 and 0.0552, respectively, on Python under both $T = 0.2$ and $T = 1.0$ temperature settings with a reasonable runtime overhead. Furthermore, it demonstrates strong cross-language generalization on Java, C++, and JavaScript, confirming its significant superiority over existing detection methods.

CCS Concepts: • **Software and its engineering**; • **Security and privacy** → **Software and application security**; • **Computing methodologies** → *Artificial intelligence*;

Additional Key Words and Phrases: Large Language Models, LLM-generated Code Detection, Zero-shot Detection, Dual-Channel Perturbation

*Both authors contributed equally to this research.
†Corresponding author.

Authors' Contact Information: Zhengdao Li, University of Science and Technology of China, Hefei, China, zhengdaoli@mail.ustc.edu.cn; Xiuwei Shang, University of Science and Technology of China, Hefei, China, shangxw@mail.ustc.edu.cn; Zhenkan Fu, University of Science and Technology of China, Hefei, China, buildxcb@mail.ustc.edu.cn; Shikai Guo, Dalian Maritime University, Dalian, China, shikai.guo@dlmu.edu.cn; Weiming Zhang, University of Science and Technology of China, Hefei, China, zhangwm@ustc.edu.cn; NengHai Yu, University of Science and Technology of China, Hefei, China, ynh@ustc.edu.cn; Kejiang Chen, University of Science and Technology of China, Hefei, China, chenkj@ustc.edu.cn.

ACM 2994-970X/2026/7-ARTFSE159
https://doi.org/10.1145/3808166

ACM Reference Format:
Zhengdao Li, Xiuwei Shang, Zhenkan Fu, Shikai Guo, Weiming Zhang, NengHai Yu, and Kejiang Chen. 2026. DualCodeDetect: Zero-Shot LLM-Generated Code Detection via Dual-Channel Perturbation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE159 (July 2026), 23 pages. https://doi.org/10.1145/3808166

1 Introduction

In recent years, the remarkable progress of LLMs, particularly code-oriented models such as CodeLlama [6, 23, 31, 47], has profoundly reshaped traditional programming paradigms and exerted a far-reaching impact on software development practices. These LLMs provide developers with valuable assistance, greatly improving coding efficiency and problem-solving capabilities [19].

However, the widespread adoption of LLMs for code generation has also raised a series of concerns regarding misuse, blurring the boundary between human-written and LLM-generated code. In educational contexts, students may exploit LLMs to complete coursework or even to gain unfair advantages in programming competitions [7]. Similarly, job applicants may rely on LLMs to pass coding assessments [8]. A recent study has shown that GPT-4 [1] achieves performance comparable to the average human level on LeetCode programming problems [5]. In industrial settings, misuse of LLMs can likewise obscure issues such as workload estimation and authorship attribution of software artifacts. Moreover, generated code may introduce security risks: Pearce et al. [30] reported in an empirical study that 40% of Copilot-generated programs contained security vulnerabilities across multiple security-critical applications. These findings underscore the need for stricter scrutiny of generated code, particularly during debugging and system verification phases.

Thus, building efficient detectors is crucial to mitigate the misuse risks of LLM-generated code. Early LLM-generated text detection studies [26] observed that LLMs tend to assign higher confidence to their own outputs, reflected in elevated log-likelihood scores. This property was exploited to effectively distinguish model-generated text from human-authored text. Researchers proposed a series of perturbation-based detection methods [4, 43], which were later naturally extended to the task of detecting LLM-generated code [34, 42, 44]. Most existing approaches follow this paradigm: they estimate the log-probability distribution of code tokens and compare the likelihood differences between the original code snippet and its perturbed variants, in order to infer its generative origin.

However, these methods originate from the context of natural language processing and do not yet fully consider the structural characteristics of the code itself. As evaluated by Pan et al. [29], typical text detectors like DetectGPT [26], GPTZero [12], and GLTR [10, 32, 39] degrade significantly on programming languages. This is fundamentally because code, unlike natural language, is constrained by strict syntactic rules and logical structures, exhibiting a "low-entropy" characteristic [14, 21]. Under the constraints of a specific programming language and context, subsequent tokens in code are often highly deterministic or have limited choice space. This characteristic introduces two main challenges. First, the perturbation space is constrained, with limited legitimate perturbation locations and replacement options, which weakens the role of probability divergence as a discriminative signal and makes it difficult to effectively amplify the differences between LLM-generated and human-written code. Second, the masking perturbations used in existing methods (e.g., DetectGPT) can easily introduce subtle yet fatal errors in code, such as improper identifier usage or syntax errors, negatively impacting the calculation of log-likelihood scores.

To address the aforementioned challenges, we propose a zero-shot detection method named **DualCodeDetect**. Through a dual-channel perturbation mechanism comprising a semantic channel and a structural channel, it better adapts to the low-entropy nature of code. While preserving code executability, it effectively amplifies the pattern differences between LLM-generated and human-written code. Figure 1 illustrates the operational workflow of DualCodeDetect. In the semantic channel, we propose an "out-of-nucleus sampling" strategy to perturb code identifiers,

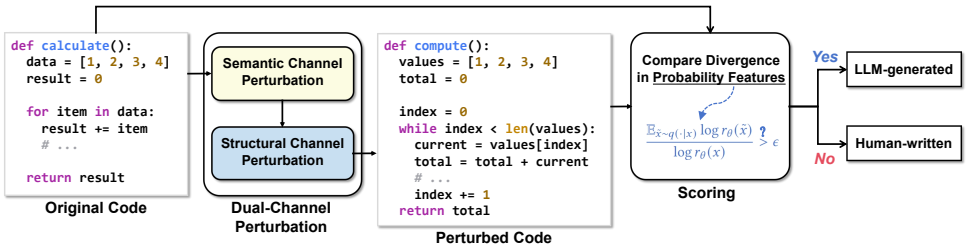


Fig. 1. The Operational Workflow of *DualCodeDetect*.

more sufficiently amplifying the perturbation differences between LLM-generated and human code. In the structural channel, we first conduct a Shannon entropy analysis on various stylistic features of code, revealing a high degree of uniformity in the structural expression of LLM-generated code. Based on this, we construct a set of predefined code structure transformation rules to perturb the syntactic representation of the code while ensuring semantic equivalence and syntactic correctness.

To evaluate the effectiveness of DualCodeDetect, we conduct a comprehensive set of experiments under both black-box and white-box detection scenarios using synthetic code generated by ten large language models across two datasets: CodeSearchNet [17] and The Stack [20]. Furthermore, ablation studies and hyperparameter analyses are carried out to systematically assess the model's robustness and the contributions of its individual components. The results indicate that DualCodeDetect achieves an average AUROC of 0.8477, alongside FPR and FNR values of 0.0430 and 0.0552, respectively, under temperature settings of $T = 0.2$ and $T = 1.0$, significantly outperforming existing state-of-the-art methods. We also perform a detailed computational overhead analysis of DualCodeDetect, and evaluate its cross-language generalization beyond Python on Java, C++, and JavaScript. Overall, its superior performance can be attributed to the collaborative perturbation mechanism of the dual channels, which, without compromising code executability, effectively overcomes the perturbation limitations imposed by the low-entropy characteristic and significantly enhances the performance of generated code detection.

Contributions. The main contributions of this paper are as follows:

- We argue that the primary limitation of existing LLM-generated code detection methods is their failure to account for the low-entropy nature of code, which hinders their adaptation to the syntactic constraints and structural features of programming languages.
- We propose a zero-shot detection method for generated code based on dual-channel perturbation. It introduces an out-of-nucleus sampling strategy in the semantic channel and incorporates code style perturbations based on Shannon entropy analysis in the structural channel, significantly improving detection performance.
- We conduct systematic experiments on multiple mainstream large language models for code and two real-world datasets, demonstrating the effectiveness of the proposed method.

2 Background and Related Works

2.1 Large Language Models for Code

Recently, both specialized and general-purpose Large Language Models (LLMs)—such as Codex [6], GPT-5.2 [35], and tools like GitHub Copilot [41]—have demonstrated remarkable capabilities in code generation. The proliferation of code LLMs is also reshaping software engineering and computer science education [7]. On one hand, these models are capable of automatically generating high-quality code, resulting in efficiency gains and personalized support for software development, teaching, and learning. On the other hand, the rapid penetration of AI-Generated Content (AIGC) has raised concerns about content authenticity and academic integrity. Particularly in educational

settings, students may use LLMs to generate homework or programming project code, thereby bypassing the process of independent thinking and skill development. This risk has driven an urgent need for AI-generated code detection technologies to maintain the fairness of assessments and the integrity of the learning process. However, existing AI-generated code detectors still face significant challenges in accuracy, robustness, and adaptability to the unique structures and semantics of code [29], motivating continued research on more effective and trustworthy detectors.

2.2 LLM-Generated Text Detection

Existing methods for detecting LLM-generated text mainly fall into two categories: **supervised learning-based methods** and **zero-shot methods**. Supervised methods train a binary classifier for discrimination using paired human-written and LLM-generated texts. Typical examples include Grover [46], OpenAI's RoBERTa-based detector [24], and Radar [16], which enhances robustness against paraphrasing attacks through adversarial training. While these methods achieve strong performance within specific domains, they often rely heavily on training data and exhibit performance degradation when generalized to unseen generative models.

Zero-shot methods require no additional training and usually detect machine generation by measuring statistical divergence from the LLM's distribution. Early approaches such as GLTR [10] uses token probabilities and ranking information to reveal patterns indicative of machine generation, and follow-up methods like LogRank [36], LRR [37], and NPR [37] refine this idea. More recently, **perturbation-based zero-shot detection** has attracted attention. Mitchell et al. [4] proposed the "probability curvature" hypothesis, suggesting that LLM-generated text is more likely to reside in negative curvature regions under local perturbations. Their mask-and-fill method perturbs the input and measures the average change in (log) probability as a detection signal. The underlying principle is that when LLM-generated text is perturbed by a masked language model, its log probability decreases significantly; whereas human-written text is less sensitive due to greater diversity. Building on this, Fast-DetectGPT [4] replaces explicit perturbation generation with conditional probability curvature, substantially reducing computational costs. DNA-GPT [43] detects generative patterns by measuring divergence across multiple completions of truncated passages, improving efficiency and generalization. Overall, perturbation-based zero-shot methods demonstrate promising cross-domain adaptability without requiring access to training data.

2.3 LLM-Generated Code Detection

Compared with text detection, research on detecting LLM-generated code remains in its nascent stages. Empirical studies [29] have demonstrated that code detection presents greater challenges, as traditional text detectors exhibit significant performance degradation on code. Ye et al. [45] determine the generative attributes by comparing the semantic similarity of code rewritten by LLMs, whereas GPTSniffer [27] fine-tunes code pre-trained models to predict code provenance. Both approaches are based on supervised learning paradigms, and their generalization capabilities are constrained by the coverage of training data. Yang et al. [44] employ surrogate models to approximate conditional probability curves, enabling curvature analysis before and after perturbation. Xu et al. [42] focus on mask perturbations in high-perplexity regions, combining global and local perplexity statistics for more robust discrimination. DetectCodeGPT [34] captures stylistic regularities in LLM-generated code through targeted perturbations of formatting symbols such as spaces and line breaks. Although some methods consider code-specific characteristics, the low-entropy distribution of code compresses the available discriminative signal space, making conventional mask perturbations struggle to amplify the differences between machine and human code. Therefore, this paper proposes a zero-shot detection method that jointly leverages semantic and structural channels of code, introducing highly discriminative perturbations without compromising executability.

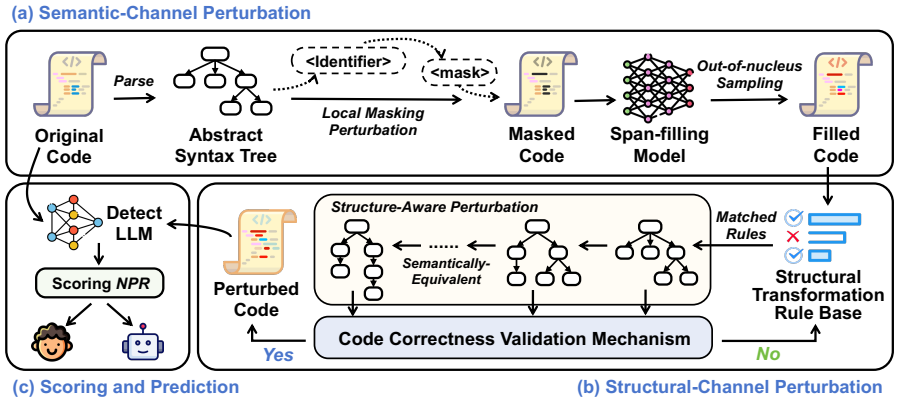


Fig. 2. The Overview of *DualCodeDetect*.

3 Problem Definition

We focus on the problem of zero-shot detection of LLM-generated code. Here, "zero-shot" means the detector is constructed without access to any labeled training data comprising both LLM-generated and human-written code. We formalize this task as a binary classification problem:

- Given a code snippet x, determine whether it was generated by a specific source model p_θ.

Following prior research on generated content detection, the detection process can be categorized into white-box and black-box scenarios:

- The **white-box scenario** assumes the detector can directly access and utilize internal information of the generative model (e.g., token probability distributions, log-likelihood values, or ranking information) to compute detection metrics.
- In contrast, the **black-box scenario** assumes the detector cannot access the target model and must approximate these metrics using a surrogate model, simulating real-world applications where the model is unknown or inaccessible.

Adopting a perturbation-based detection approach, we define a perturbation process $q(\cdot \mid x)$ that transforms x into an equivalent form $\widetilde{x}$. We hypothesize that if x was written by an LLM, its discriminative score will exhibit a significant decrease after perturbation. The core challenges lie in designing an effective perturbation strategy $q(\cdot \mid x)$ and computing a reliable discriminative score.

4 Methodology

4.1 Overview

Based on prior empirical studies [34] comparing LLM-generated and human-written code, as well as our own empirical observations (§4.3), we identify consistent differences between them in both identifier selection and structural expression. Specifically, LLMs show a stronger preference for identifiers with higher predicted probabilities and exhibit greater uniformity in structural patterns, whereas human-written code exhibits greater diversity and variability in both aspects.

Therefore, this paper proposes **DualCodeDetect**, a dual-channel perturbation framework for zero-shot detection of LLM-generated code, which simultaneously applies perturbations through both semantic and structural channels. In the semantic channel, it targets identifier usage patterns to disrupt the LLM's high-probability preference in identifier selection. In the structural channel, it focuses on code structure and stylistic patterns to diminish the structural consistency characteristic of LLM-generated code. This approach significantly amplifies the statistical distribution differences between LLM-generated and human-written code while preserving executability and semantic integrity. Following dual-channel perturbation, we compute perturbation probability features from

Algorithm 1 Semantic Channel Perturbation

Input: Original code x; Span-filling model $\mathcal{M}$; Nucleus threshold p; Sample size k
Output: Semantically perturbed code $\widetilde{x}_s$

AST $\leftarrow$ tree-sitter.parse(x) ▷ Parse the code to AST
identifiers $\leftarrow$ extract_identifiers(AST) ▷ Locate all identifier nodes
$id_{\text{mask}} \leftarrow$ sample(identifiers, 1) ▷ Randomly sample one identifier
$x_{\text{mask}} \leftarrow$ replace($x, id_{\text{mask}}, \langle\text{extra\_id}\rangle$) ▷ Mask the selected identifier
$\mathcal{V}_{\text{out}} \leftarrow$ get_out_of_nucleus_set($\mathcal{M}, x_{\text{mask}}, p$) ▷ See Eq. (1) & (2)
$S \leftarrow$ sample_subset($\mathcal{V}_{\text{out}}, k$) ▷ Uniformly sample a subset of size k from $\mathcal{V}_{\text{out}}$
$v_{\text{fill}} \leftarrow$ sample(S, 1) ▷ Randomly select a token from S
$\widetilde{x}_s \leftarrow$ replace($x_{\text{mask}}, \langle\text{extra\_id}\rangle, v_{\text{fill}}$) ▷ Fill the mask with the selected token
return $\widetilde{x}_s$

both original and perturbed code samples, and employ a decision threshold to determine whether the code was generated by an LLM. The overview of DualCodeDetect is illustrated in Figure 2, consisting of three main stages: (a) applying semantic-channel perturbation, (b) performing structural-channel perturbation, and (c) scoring the Normalized Perturbed Rank (NPR) for classification.

4.2 Semantic Channel Perturbation

4.2.1 Empirical Basis. Through a statistical analysis of syntactic element distributions in code corpora, Shi et al. [34] found that identifiers account for approximately 40% of all syntactic elements. Their high frequency in code implies greater perturbability potential without compromising grammatical integrity or executability. Furthermore, it was empirically found that LLM-generated code demonstrates a significantly high-probability preference in identifier selection, whereas human-written code exhibits greater diversity in identifier naming.

4.2.2 Semantic Perturbation Strategy. Therefore, in the semantic channel, we design a *local masking perturbation mechanism* on identifiers, combined with an out-of-nucleus sampling strategy for filling. This ensures that the substituted identifiers exhibit low predicted probabilities within the original context. Since such perturbations disrupt the "high-probability consistency" of LLMs in identifier selection, the divergence in statistical probability features before and after perturbation is greatly amplified in LLM-generated code, while remaining relatively stable in human-written code.

Specifically, given an input code snippet x, we first construct its Abstract Syntax Tree (AST) using the tree-sitter parser [1] and locate all identifier nodes (e.g., variable names, function names, parameter names). In each perturbation, we randomly sample one identifier and replace it with a placeholder ⟨extra_id_n⟩, obtaining a masked code sequence x_{mask}. During the filling phase, a pre-trained span-filling model (CodeT5 [40] in this work) is utilized to recover the identifier at the placeholder position. The process of the semantic channel perturbation is shown in Algorithm 1.

Unlike conventional top-p nucleus sampling [15], we adopt an out-of-nucleus sampling strategy. As shown in Figure 3, the out-of-nucleus sampling strategy avoids selecting high-probability candidate tokens during the mask-filling process, whereas large language models (LLMs) exhibit a strong preference for choosing high-probability tokens within the nucleus when generating code. In contrast, human-written code does not show such a tendency. Consequently, after applying out-of-nucleus sampling perturbations, LLM-generated code exhibits more substantial divergence in its statistical characteristics, while human-written code maintains relatively minor statistical fluctuations. Compared to conventional sampling approaches, this strategy further amplifies the

[1] https://github.com/tree-sitter/tree-sitter

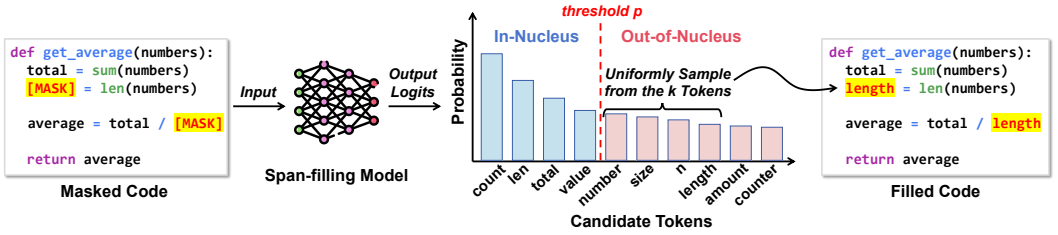


Fig. 3. Details of *Out-of-Nucleus Sampling*.

distinction in statistical features between LLM-generated and human-written code. Let the generative distribution be $P_{\mathcal{M}}(\cdot \mid x_{\text{mask}})$. We first remove the in-nucleus set $\mathcal{V}_p$, which contains candidate tokens whose cumulative probability from highest to lowest does not exceed a threshold p:

$$\mathcal{V}_p = \left\{ v_i \in \mathcal{V} \;\middle|\; \sum_{j=1}^{i} P_{\mathcal{M}}(v_j \mid x_{\text{mask}}) \leq p \right\} \tag{1}$$

The corresponding out-of-nucleus set is defined as:

$$\mathcal{V}_{\text{out}} = \mathcal{V} \setminus \mathcal{V}_p \tag{2}$$

Subsequently, we select the fill token in two steps from the out-of-nucleus set: first, uniformly sample a subset of size k from $\mathcal{V}_{\text{out}}$:

$$S \subseteq \mathcal{V}_{\text{out}}, |S| = k \tag{3}$$

then randomly select a token from the subset S as the fill value.

Formally, given a code snippet x, we define the semantic channel perturbation operator $\mathcal{P}_{\text{semantic}}(\cdot)$, which yields the perturbed code snippet $\widetilde{x}_s$ after identifier masking and out-of-nucleus sampling:

$$\widetilde{x}_s = \mathcal{P}_{\text{semantic}}(x) \tag{4}$$

This strategy ensures that the fill value has a low generation probability in the original context, thereby amplifying the divergence in statistical probability features of LLM-generated code before and after perturbation, while introducing relatively minor fluctuations in human-written code.

4.3 Structural Channel Perturbation

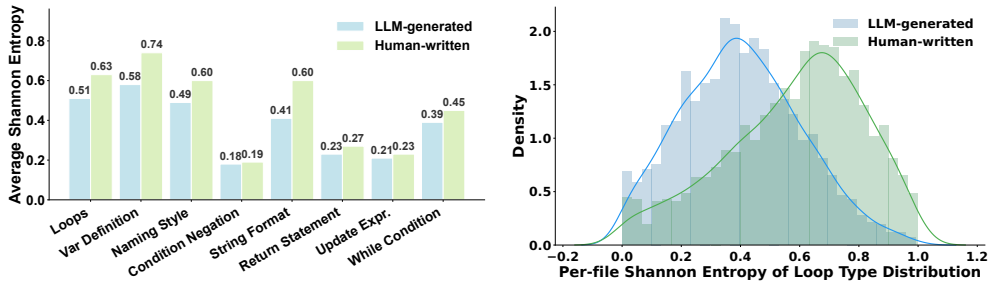
4.3.1 Empirical Analysis . To further reveal the diversity differences in structural expression between LLM-generated code and human code, we conduct statistical analyses on various stylistic features of code, using Shannon entropy [33] as the measurement metric. Specifically, for a given structural feature (e.g., loop type, naming style), we define its set of possible candidate categories (detailed in Table 1). For a single code x, the Shannon entropy for this feature is defined as:

Table 1. Candidate Set Definition for Structural Features

Structural Feature	Candidate Set
Loops	{`for` loop, `while` loop}
Variable Definition	{defined at function start, at first usage}
Naming Style	{camelCase, PascalCase, snake_case, _underscore_init}
Condition Negation	{`if condition`, `if not (condition)`}
String Format	{f-string, `.format()`}
Return Statement	{implicit `None`, explicit `return None`}
Update Expression	{`i += 1`, `i = i + 1`}
While Condition	{`while condition`, `while True:` / `while 1`}

$$H(x) = -\sum_{i=1}^{k} p_i \log p_i \tag{5}$$

where p_i represents the relative frequency of the snippet x on the i-th candidate category of this feature, and k is the total number of candidate categories for this feature. Subsequently, we compute



(a) Shannon Entropy for Various Structural Features (b) The distribution of different loop types is presented.

Fig. 4. Divergence in Structural Diversity Between LLM-Generated Code and Human-Written Code. Figure (a) compares the average Shannon entropy of LLM-generated code and human-written code across various structural features. Figure (b) uses loop structures as a specific example to contrast the distribution patterns of loop types between LLM-generated code and human-written code.

the entropy value for all code samples in the corpus and take the average as the overall diversity metric for the feature.

As shown in Figure 4(a), the average Shannon entropy of LLM-generated code is significantly lower than that of human code across almost all structural features, indicating a tendency towards more uniform and consistent patterns in structural expression, while human-written code exhibits greater diversity. Furthermore, we conduct a detailed analysis using loop structures as an example. The results in Figure 4(b) show that the entropy distribution of LLM-generated code is markedly concentrated in low-value regions, whereas the distribution of human code is more dispersed, suggesting that the latter demonstrates greater variety in the choice of looping styles.

4.3.2 Structural Perturbation Strategy. Based on the above empirical analysis, we further propose structural channel perturbations on code to amplify the differences between LLM-generated code and human-written code at the stylistic and structural levels. Unlike the semantic channel, which primarily focuses on local replacements of identifiers, the structural channel targets diversity in code structure and expression patterns. The core hypothesis is that due to the high degree of homogeneity exhibited by LLMs in syntactic and stylistic choices (such as preferred loop types, conditional statement forms, or naming conventions), when code undergoes semantically equivalent probabilistic structural perturbations, the divergence in statistical features of LLM-generated code will be significantly greater than that of human-written code.

Formally, given the code snippet $\widetilde{x}_s$ perturbed via the semantic channel, we define a structural perturbation operator $\mathcal{P}_{\text{structural}}(\cdot)$ that performs semantic-preserving structural transformations:

$$\widetilde{x} = \mathcal{P}_{\text{structural}}\left(\widetilde{x}_s\right) \tag{6}$$

Semantically-Equivalent Structural Transformation Rule Base. To implement this operator, we predefine a structural transformation rule base $\mathcal{R} = \{R_1, R_2, \ldots, R_m\}$, which serves as a collection of semantics-preserving structural rewriting rules, as shown in Table 2. Each rule R_i defines a transformation from a source syntactic pattern to a target syntactic pattern. This rule base covers common syntactic structures in Python, such as loop structure transformations R_{Loop} : `for` $\leftrightarrow$ `while` and conditional negation transformations R_{CondNeg} : `if condition` $\leftrightarrow$ `if not (condition)`. Theoretically, all rules guarantee semantic equivalence while introducing differences in code form.

Structure-Aware Perturbation Mechanism. During the rule application process, we adopt a *structure-aware perturbation mechanism*. Given the semantically perturbed code snippet $\widetilde{x}_s$, the system first parses its Abstract Syntax Tree (AST) by tree-sitter and identifies internally matchable

Algorithm 2 Structural Channel Perturbation

Input: Semantically perturbed code $\widetilde{x}_s$; Rule base $\mathcal{R} = \{R_1, R_2, ..., R_m\}$
Output: Structurally perturbed code $\widetilde{x}$

AST $\leftarrow$ tree-sitter.parse($\widetilde{x}_s$)
$C \leftarrow \{R_i \in \mathcal{R} \mid R_i \text{ matches AST}\}$ ▷ Find all applicable rules
Group rules in C into Mutually Exclusive Groups $\{G_1, G_2, ...\}$
$\widetilde{x} \leftarrow \widetilde{x}_s$
for each mutually exclusive group G in C **do**
    $r^* \sim \text{Uniform}(G)$ ▷ Uniformly sample a rule from the group
    $\widetilde{x}_{\text{temp}} \leftarrow r^*(\widetilde{x})$ ▷ Apply the transformation
    if is_syntactically_valid($\widetilde{x}_{\text{temp}}$) **then** ▷ AST validation
        $\widetilde{x} \leftarrow \widetilde{x}_{\text{temp}}$
    end if
end for
return $\widetilde{x}$

patterns, obtaining a candidate set of applicable transformation rules:

$$C(\tilde{x}_s) = \{R_i \in \mathcal{R} \mid R_i \text{ matches } \tilde{x}_s\} \tag{7}$$

The system then selects and applies perturbation rules from the candidate set $C(\tilde{x}_s)$. To avoid logical conflicts, the rules are categorized into Mutually Exclusive Groups and Independent Rules:

(1) Mutually Exclusive Group: Multiple equivalent forms of the same syntactic unit form a group, such as loop structures `for`, `while`. The system uniformly samples one form from the group:

$$r \sim \text{Uniform}(G), \widetilde{x} = r(\widetilde{x}_s) \tag{8}$$

where G denotes this mutually exclusive group. If the sampled form is identical to the original, the code remains unchanged; otherwise, the transformation is applied.

(2) Independent Rules: These refer to rules that operate on different syntactic units and do not conflict with each other (e.g., loop transformation and update expression transformation). These rules correspond to independent parallel applications of multiple mutually exclusive groups. The system sequentially samples them and decides whether to trigger the transformation. Thus, the final perturbation result may be a combination of multiple rules:

$$\widetilde{x} = r^{(k)}\left(\widetilde{x}_{k-1}\right), \widetilde{x}_{k-1} = r^{(k-1)}\left(\widetilde{x}_{k-2}\right), \cdots, \widetilde{x}_1 = r^{(1)}\left(\widetilde{x}_s\right) \tag{9}$$

where $k = |C\left(\widetilde{x}_s\right)|$. It's worth emphasizing that syntax errors in the input code do not render the structural channel entirely ineffective. This is because the tree-sitter parser is specifically designed for robustness and error recovery: even when syntax errors are present, it can isolate faults as ERROR nodes while correctly parsing the rest. Since structure-aware perturbation operates via local pattern matching, it bypasses problematic regions and applies transformations to valid code segments. Therefore, the structural channel does not collapse; rather, only the number of applicable transformation rules k decreases accordingly.

Perturbed Code Correctness Validation Mechanism. Despite the theoretical equivalence of our rigorous syntax tree transformations, empirical results from practical implementation show that 3 out of the 14 rules occasionally trigger syntax parsing errors in fewer than 1% of cases, while the remaining 11 rules are consistently error-free. Therefore, after each transformation step, DualCodeDetect performs syntactic validity verification using an AST parser to ensure that no additional syntax errors are introduced into the perturbed code. If a transformation introduces a syntax error, the system rolls back and resamples other candidate rules. Finally, after a series of structural rewrites, we obtain a semantically equivalent but structurally diversified code version $\widetilde{x}$.

Table 2. Code Transformation Types and Descriptions.

Type	Transformation	Description
Stylistic & API Conventions	(1) Naming Style	Switch between camelCase, PascalCase, snake_case and _underscore_init
	(2) Dictionary Access	Dictionary access conversion: `dict[key]` ↔ `dict.get(key)`
	(3) String Format	String formatting conversion: f-string ↔ `.format()` method
	(4) Return Statement	Function return conversion: implicit `None` ↔ explicit `return None`
Structural Statement Transformations	(5) Variable Initialization	Switch between single assignment and two-step assignment, e.g., `var=expression()` ↔ `var=None;var=expression()`
	(6) Variable Definition Position	Move assignment statements within function body, e.g., move `x=10` from middle of function to beginning or to first use of `x`
	(7) Update Expression	Increment expression style conversion, e.g., `i+=1` ↔ `i=i+1`
	(8) Loop Type	Loop structure conversion: `for` loop ↔ `while` loop
	(9) While Loop Condition	Convert while condition to always-true literal `True` or `1`, e.g., `while condition:` ↔ `while True:` or `while 1:` … `break`
	(10) Ternary Expression	Ternary operator expansion and merge: `x = a if b else c` ↔ `if b: \n x = a \n else: \n x = c`
Conditional Logic Transformations	(11) Nested Conditions	Interchange between compound conditions and nested if statements, e.g., `if a: \n if b:` ↔ `if a and b:`
	(12) Block Swap	Negate condition and swap `then` and `else` code blocks of `if` statement
	(13) Condition Negation	Condition negation: `if condition` ↔ `if not (condition)`, and swap code blocks
	(14) Condition Chain/Switch	Interchange between if-elif-else condition chain and switch statement: `if-elif-else` ↔ `dict` lookup

The detailed procedure for structural channel perturbation is outlined in Algorithm 2. The strategy overcomes the perturbation limitations imposed by low-entropy constraints, causing LLM-generated code to exhibit greater statistical feature fluctuation under structural transformations compared to human code, thereby providing more discriminative signals for the detection task.

4.4 Scoring and Prediction

In prior perturbation-based zero-shot detection methods, such as Fast-DetectGPT [4] and DetectGPT [26], the difference in token log-likelihood is commonly adopted as the metric to measure perturbed probability features for distinguishing between LLM-generated and human-written content. In subsequent research, Su et al. [37] discover that compared to log-likelihood values, the log-rank of LLM-generated text is more sensitive to minor perturbations. They propose the Normalized Perturbed Rank (NPR) as a metric, which significantly outperforms traditional log-likelihood difference in detection performance.

Therefore, we employ the NPR score as the key metric for evaluating perturbation probability features, aiming to enhance the discrimination between generated code and human-written code. The NPR score is calculated as follows:

$$\mathrm{NPR}(x, p_\theta, q) = \frac{\mathbb{E}_{\tilde{x}\sim q(\cdot|x)} \log r_\theta(\tilde{x})}{\log r_\theta(x)} \tag{10}$$

where the perturbed code segment $\widetilde{x}$ is the result of applying both semantic-channel and structural-channel perturbations to the given code segment x, and $\log r_\theta(x)$ denotes the logarithm of the rank of segment x sorted by likelihood under model p_θ.

Finally, by comparing the NPR score of the code segment in question with a preset threshold, we can determine whether the code segment was generated by an LLM or written by a human.

5 Evaluation

Our experiments are designed to answer the following **Research Questions (RQs)**:

- **RQ1:** How effective is the DualCodeDetect method at distinguishing between LLM-generated code and human-written code in a white-box setting?
- **RQ2:** What is the detection performance of the DualCodeDetect method in a black-box setting?

- **RQ3:** What is the computational overhead of DualCodeDetect compared with baseline methods?
- **RQ4:** How does each perturbation channel impact the performance of DualCodeDetect?
- **RQ5:** How do the parameter settings of DualCodeDetect affect its detection performance?
- **RQ6:** How do code attributes influence the effectiveness of DualCodeDetect?

5.1 Experimental Setup

Datasets. To comprehensively evaluate the performance of our proposed detection method, we conduct experiments on two widely-used and representative code datasets: CodeSearchNet [17] and The Stack [20]. Unlike benchmarks comprising simplified programming problems such as HumanEval [6] or MBPP [3], these two datasets consist of code sourced from diverse real-world projects, enabling the analysis of disparities between human-written and LLM-generated code across a broader spectrum of practical applications.

- **CodeSearchNet** [17] is a curated dataset designed for code retrieval and semantic search. Its composition of code gathered from open-source projects ensures that the samples reflect realistic coding scenarios and conventions.
- **The Stack** [20] offers a massive collection of source code from diverse open-source repositories. This dataset provides a vast scale and diversity, further testing the generalization capability of detection methods across a wide array of coding styles and domains.

All primary experiments are conducted on Python code. The data is constructed by concatenating function definitions with their corresponding docstrings to form natural language prompts, which are then fed into various code LLMs to generate synthetic code samples. For each combination of dataset and LLM, we generate 10,000 pairs of human-written and LLM-generated code, which are used for the empirical analysis in Section 4.3.1. From this pool, we randomly sample 500 instances for each evaluation. The maximum length of each code sample is truncated to 128 tokens to standardize input size. Furthermore, to ensure a fairer evaluation and to focus the detection on the genuine code structure rather than on potentially revealing commenting patterns, all comments are rigorously removed from every code sample in the test set prior to assessment.

Hardware Environment. A workstation equipped with Intel Xeon Gold 6330 CPU and eight NVIDIA A6000 GPUs (48 GB VRAM each).

Generation Models. The LLMs utilized for generating machine code in our study comprise Incoder-1B [9], Phi-1 [11], StarCoder-3B [22], WizardCoder-3B-V1.0 [25], CodeGen2-3.7B [28], CodeLlama-7b-Instruct-hf [31], DeepSeek-Coder-V2-Lite-Instruct [48], and Qwen3-Coder-30B-A3B-Instruct [38]. The model sizes range from 1.3 billion to 30 billion parameters. Furthermore, we incorporate two closed-source models: GPT-5.2 [35] and Claude-4.5-Sonnet [2], which are specifically employed to validate generalization and effectiveness of DualCodeDetect in black-box scenarios, as their internal logits or token probabilities are inaccessible.

Baselines. To ensure a comprehensive comparison, we evaluate our method against several state-of-the-art and representative baselines for LLM-generated code detection:

- **Log p(x)** [10]: Computes the average token-wise log-probability under the source model, where higher scores suggest LLM-originated content.
- **Entropy** [18]: Measures the average predictive entropy of each token, with lower entropy indicating LLM-generated code.
- **Rank / Log Rank** [36]: Uses the average (log) rank of tokens, under the assumption that LLM-generated text exhibits lower ranks.
- **DetectGPT** [26]: Calculates the log-probability gap between the original code and its perturbed versions to discriminate LLM-generated content.
- **DetectLLM** [37]: Incorporates two variants: LRR (combining log-likelihood and log-rank) and an NPR-enhanced version of DetectGPT.

- **GPTSniffer** [27]: A supervised classifier fine-tuned on CodeBERT to distinguish human-written from LLM-generated code samples.
- **DetectCodeGPT** [34]: Perturbs code by altering whitespace and compares log-rank divergence before and after perturbation.

Evaluation Metric. Following the established methodology of Mitchell et al. [26], we use the Area Under the Receiver Operating Characteristic Curve (AUROC) to evaluate the performance of all detectors. Given a sequence of True Positive Rate (TPR) and False Positive Rate (FPR) under different thresholds, the AUROC can be formally defined as:

$$\text{AUROC} = \int_0^1 \text{TPR}(t) \ \ dt, \tag{11}$$

where t denotes the classification threshold. This metric provides a comprehensive assessment of model performance across all possible decision thresholds, thereby exhibiting threshold-invariant characteristics. We further introduce False Positive Rate (FPR) and False Negative Rate (FNR) as complementary metrics to provide a more granular analysis of detection errors. For a fair comparison across methods, we select the best decision threshold for each detector by maximizing Youden's index (TPR - FPR), and compute the corresponding FPR and FNR at this optimal threshold.

5.2 RQ1: Detection Performance in a White-Box Scenario

This experiment systematically evaluates the detection performance of DualCodeDetect alongside nine baseline methods on code generated by eight open-source LLMs of varying parameter sizes, using two widely adopted code datasets: CodeSearchNet and The Stack. As shown in Table 3, across diverse generation scenarios with different temperature settings ($T = 0.2$ / $T = 1.0$), DualCodeDetect achieves the best performance in 27 out of all 32 dataset-model combinations, attaining an average AUROC of 0.8477 and outperforming all baseline methods overall, while also yielding lower error rates with an average FPR of 0.1298 and FNR of 0.1556 at the selected decision threshold.

To comprehensively compare the characteristics and limitations of the baseline methods, we categorize them into three groups: (1) **Supervised methods** (e.g., GPTSniffer), which perform well under known model conditions but exhibit limited generalizability to unseen models; (2) **Zero-shot statistical methods** (e.g., Log-Likelihood, Entropy, Rank), whose performance deteriorates substantially when dealing with large-scale models; and (3) **Perturbation-based zero-shot methods** (e.g., DetectGPT, NPR, DetectCodeGPT), which represent the closest competitors to our approach. Across all three categories, DualCodeDetect consistently demonstrates stable and superior performance. Unlike DetectGPT and NPR, which rely solely on semantic random perturbations, DualCodeDetect not only strengthens targeted perturbations on identifiers but also introduces structural perturbations, enabling the model to capture both semantic discrepancies and the stylistic uniformity inherent in LLM-generated code. This dual-channel design amplifies statistical distribution differences and significantly enhances discriminative capability. Compared with the strongest baseline, DetectCodeGPT, DualCodeDetect improves the average AUROC by 3.13%; Notably, the consistent reductions in both FPR and FNR suggest that these gains go beyond ranking metrics, yielding fewer false alarms and fewer missed detections.

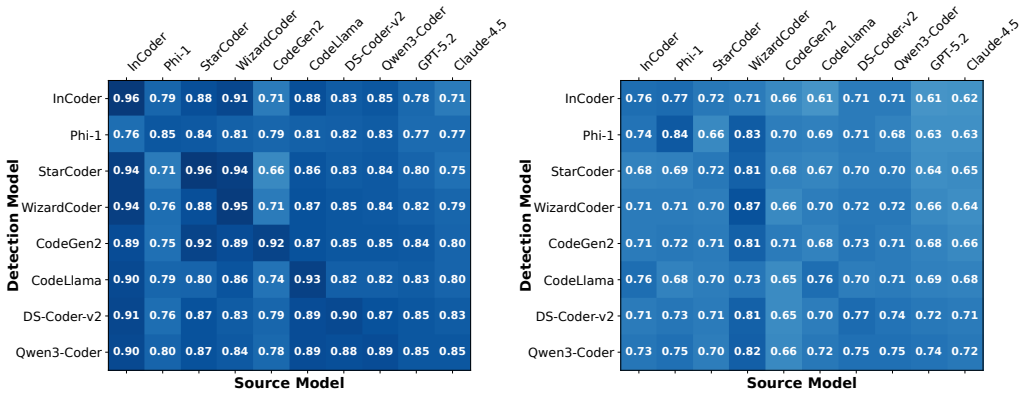
From the perspective of model parameter scale, DualCodeDetect effectively mitigates the model size issue prevalent in existing detection approaches. Traditional statistical methods like Rank and Log Rank, exhibit significant performance degradation when model parameters exceed 3B. In contrast, DualCodeDetect maintains an AUROC of 0.8944 even on code generated by the 30B parameter Qwen3-Coder model using the The Stack dataset, demonstrating strong robustness across generative models of different scales. Regarding the temperature parameter, increasing it from 0.2 to 1.0 elevates generation randomness, resulting in more diverse model outputs and

Table 3. Detection Performance Comparison of Different Methods in a White-Box Scenario. (Due to space limitations, each cell reports only AUROC, while FPR and FNR are reported as averages over all Code LLMs)

Dataset	Code LLM	Detection Methods									
		log p(x)	Entropy	Rank	Log Rank	DetectGPT	LRR	NPR	GPTSniffer	DetectCodeGPT	**DualCodeDetect**
CodeSearchNet ($T = 0.2$)	Incoder (1.3B)	0.9712	0.1085	0.8782	0.9795	0.4832	0.9598	0.8231	0.9328	**0.9802**	0.9612
	Phi-1 (1.3B)	0.7982	0.4218	0.6502	0.7618	0.7112	0.4124	0.7668	0.3958	0.8211	**0.8523**
	StarCoder (3B)	0.9208	0.3045	0.7688	0.9442	0.6849	0.9147	0.8917	0.7815	0.9493	**0.9615**
	WizardCoder (3B)	0.9179	0.3032	0.7659	0.9222	0.6552	0.8078	0.8779	0.7536	0.9331	**0.9502**
	CodeGen2 (3.7B)	0.7129	0.4514	0.7429	0.7299	0.6153	0.8099	0.6279	0.5429	0.8758	**0.9181**
	CodeLlama (7B)	0.8952	0.3276	0.7367	0.9118	0.8014	0.8434	0.5992	0.7598	0.9184	**0.9311**
	DS-Coder-V2 (16B)	0.8712	0.3952	0.6801	0.8331	0.7632	0.6961	0.7272	0.6439	0.8821	**0.9016**
	Qwen3-Coder (30B)	0.8453	0.3844	0.7003	0.8115	0.7417	0.6965	0.6711	0.6201	0.8743	**0.8931**
	Avg. AUROC ↑	0.8666	0.3371	0.7404	0.8618	0.6820	0.7676	0.7481	0.6788	0.9043	**0.9211**
	Avg. FPR ↓	0.1156	0.6389	0.2476	0.1243	0.3124	0.2149	0.2317	0.3024	0.0542	**0.0451**
	Avg. FNR ↓	0.1481	0.6912	0.2738	0.1426	0.3375	0.2477	0.2584	0.3331	0.0697	**0.0583**
CodeSearchNet ($T = 1.0$)	Incoder (1.3B)	0.7826	0.4269	0.7899	**0.7978**	0.6359	0.7529	0.6903	0.6863	0.7799	0.7622
	Phi-1 (1.3B)	0.6619	0.4689	0.5809	0.6399	0.7394	0.4629	0.8014	0.4259	0.8276	**0.8434**
	StarCoder (3B)	0.6675	0.4945	0.7088	0.6923	0.6606	0.7151	0.6852	0.6399	0.7014	**0.7169**
	WizardCoder (3B)	0.8419	0.3465	0.7374	0.8439	0.6073	0.7066	0.7617	0.7169	0.8367	**0.8721**
	CodeGen2 (3.7B)	0.4585	0.6364	0.6685	0.4733	0.4898	0.5631	0.5309	0.4125	0.6882	**0.7069**
	CodeLlama (7B)	0.6564	0.4956	0.6859	0.6757	0.6524	0.6869	0.6616	0.6543	0.7194	**0.7603**
	DS-Coder-V2 (16B)	0.6024	0.4771	0.6225	0.6231	0.4810	0.6825	0.7093	0.6413	0.7331	**0.7672**
	Qwen3-Coder (30B)	0.6153	0.4622	0.6368	0.5910	0.4519	0.6881	0.6725	0.6204	0.7145	**0.7493**
	Avg. AUROC ↑	0.6608	0.4760	0.6788	0.6671	0.5898	0.6573	0.6891	0.5997	0.7501	**0.7723**
	Avg. FPR ↓	0.3192	0.5059	0.2314	0.3152	0.4003	0.3259	0.2918	0.3812	0.2353	**0.2114**
	Avg. FNR ↓	0.3624	0.5417	0.3389	0.3642	0.4195	0.3691	0.3265	0.4165	0.2607	**0.2398**
The Stack ($T = 0.2$)	Incoder (1.3B)	0.9595	0.1498	0.8848	0.9614	0.6162	0.9539	0.8672	0.9193	**0.9699**	0.9696
	Phi-1 (1.3B)	0.8152	0.4419	0.6867	0.7723	0.7197	0.4123	0.8207	0.4741	0.8526	**0.8826**
	StarCoder (3B)	0.9199	0.3178	0.7944	0.9330	0.6925	0.9136	0.9135	0.7816	0.9351	**0.9492**
	WizardCoder (3B)	0.9127	0.3297	0.8064	0.9111	0.6486	0.7843	0.8675	0.7895	0.9187	**0.9465**
	CodeGen2 (3.7B)	0.7272	0.4152	0.8031	0.7402	0.5389	0.7705	0.5771	0.4621	0.8589	**0.8829**
	CodeLlama (7B)	0.8677	0.3666	0.7467	0.8894	0.7988	0.8459	0.5537	0.7720	0.8827	**0.9064**
	DS-Coder-V2 (16B)	0.8527	0.4467	0.7081	0.8384	0.7795	0.6849	0.7466	0.6620	0.8627	**0.9147**
	Qwen3-Coder (30B)	0.8333	0.3905	0.7106	0.8215	0.7350	0.7034	0.6822	0.6184	0.8556	**0.8944**
	Avg. AUROC ↑	0.8610	0.3573	0.7676	0.8584	0.6912	0.7586	0.7536	0.6849	0.8920	**0.9183**
	Avg. FPR ↓	0.1127	0.5974	0.2117	0.1217	0.2795	0.2591	0.2246	0.2918	0.0895	**0.0630**
	Avg. FNR ↓	0.1385	0.6798	0.2462	0.1541	0.3250	0.2673	0.2508	0.3145	0.1154	**0.0871**
The Stack ($T = 1.0$)	Incoder (1.3B)	0.7411	0.4692	0.7774	0.7656	0.6225	0.7547	0.6888	0.6947	**0.7895**	0.7721
	Phi-1 (1.3B)	0.7942	0.4306	0.6767	0.7576	0.6819	0.4207	0.7856	0.5085	0.8423	**0.8756**
	StarCoder (3B)	0.6434	0.5126	0.7111	0.6709	0.5997	0.7181	0.6739	0.7344	0.6957	**0.7412**
	WizardCoder (3B)	0.8394	0.3560	0.7585	0.8324	0.6478	0.6537	0.8030	0.7867	0.8325	**0.8730**
	CodeGen2 (3.7B)	0.4917	0.6147	0.5732	0.5057	0.4438	0.5841	0.5279	0.4366	0.6684	**0.7016**
	CodeLlama (7B)	0.6029	0.5361	0.6552	0.6192	0.6217	0.6466	0.6327	**0.7595**	0.6738	0.7268
	DS-Coder-V2 (16B)	0.6291	0.5074	0.6375	0.6170	0.5013	0.6522	0.6851	0.6581	0.7214	**0.7884**
	Qwen3-Coder (30B)	0.6377	0.4928	0.6512	0.5884	0.4709	0.6722	0.6641	0.6125	0.7084	**0.7545**
	Avg. AUROC ↑	0.6724	0.4899	0.6801	0.6696	0.5737	0.6378	0.6826	0.6489	0.7415	**0.7792**
	Avg. FPR ↓	0.3026	0.4937	0.2991	0.3080	0.4007	0.3458	0.2995	0.3255	0.2336	**0.1996**
	Avg. FNR ↓	0.3477	0.5216	0.3378	0.3362	0.4295	0.3892	0.3318	0.3697	0.2771	**0.2371**
Average AUROC ↑		0.7652	0.4151	0.7167	0.7642	0.6342	0.7053	0.7184	0.6531	0.8220	**0.8477**
Average FPR ↓		0.2125	0.5590	0.2475	0.2173	0.3482	0.2864	0.2619	0.3252	0.1532	**0.1298**
Average FNR ↓		0.2492	0.6086	0.2992	0.2493	0.3779	0.3183	0.2919	0.3585	0.1807	**0.1556**

a corresponding significant increase in detection difficulty. Although DualCodeDetect exhibits certain performance fluctuations at $T = 1.0$, it still consistently outperforms all other compared methods.

To further validate the statistical significance of the performance differences, we repeat each experiment ten times under the same settings, and apply the Wilcoxon rank-sum test to the AUROC distributions of DualCodeDetect and each baseline. The results show that the differences between DualCodeDetect and all baselines are statistically significant (p-values < 0.01), indicating that the observed differences are unlikely to be caused by random fluctuations. Furthermore, we use Cliff's Delta to measure the effect size. As shown in Table 3, cells without background color indicate that DualCodeDetect not only outperforms the corresponding baseline in terms of AUROC, but also achieves a medium or large effect size, suggesting that the improvement is practically meaningful;



(a) Temperature $T = 0.2$ (b) Temperature $T = 1.0$

Fig. 5. Cross-model Detection Performance (AUROC) in a Black-Box Scenario. (on CodeSearchNet Dataset)

yellow cells indicate that DualCodeDetect is better but with a small effect size, implying that the advantage is relatively modest. Notably, in all cases where DualCodeDetect performs worse than a baseline, the effect sizes are consistently small (highlighted in red), which suggests that these underperforming cases do not correspond to substantial performance gaps. Taken together, the significance tests and effect-size results demonstrate that DualCodeDetect achieves higher AUROC in most dataset-model combinations with practically meaningful gains, while in the few settings where it does not dominate, the performance drop is limited to a small effect size, further indicating the robustness of DualCodeDetect across different configurations.

Answering RQ1: Experimental results demonstrate that DualCodeDetect achieves an average AUROC of 0.8477 across two datasets, eight open-source code LLMs, and two temperature settings, and it also yields low error rates with an average FPR of 0.1298 and FNR of 0.1556, outperforming all baseline methods with a significant improvement in detection performance.

5.3 RQ2: Detection Performance in a Black-Box Scenario

Section 5.2 evaluates the detection performance of DualCodeDetect in white-box scenarios. As described in §3, the objective of the task is to determine whether a given code snippet x originates from a specific source model p_θ. However, in practical environments, the code under examination could be either human-written or generated by any of multiple different LLMs. Pre-specifying a particular source model p_θ prior to detection is often impractical, making it impossible to determine which LLM should be used to compute the Normalized Perturbation Rank (NPR), thereby limiting the applicability of the detection method. Therefore, to better align with real-world requirements, we further conduct experiments under black-box scenarios by introducing a surrogate model mechanism: when the true source model is unknown (or inaccessible), we use another LLM as a proxy to estimate NPR scores and conduct detection, thereby evaluating the cross-model generalization capability of DualCodeDetect.

As in §5.2, experiments are carried out on the CodeSearchNet dataset at temperature settings of $T = 0.2$ and $T = 1.0$, covering eight open-source code LLMs: Incoder, Phi-1, StarCoder, WizardCoder, CodeGen2, CodeLlama, DeepSeek-Coder-V2, and Qwen3-Coder, as well as two closed-source LLMs, i.e., GPT-5.2 and Claude-4.5-Sonnet (used only as source models rather than detection models due to inaccessible logits), to evaluate cross-model detection performance. Due to space limitations, we only report AUROC, and the conclusions remain the same when using FPR or FNR.

Table 4. Computational Cost (in seconds) of Different Methods. ($T = 0.2$, on CodeSearchNet Dataset)

Category	Methods	Logits Calculation for Original Code	Perturbation	Logits Calculation for Perturbed Code	Total Cost
Zero-shot Statistical	Log Rank	129	\	\	**129**
Perturbation-based Zero-shot	DetectGPT	129	2,197	12,913	**15,239**
	DetectCodeGPT	129	46	2,579	**2,754**
	DualCodeDetect	129	402 + 117	2,616	**3,264**

As shown in Figure 5, DualCodeDetect exhibits differentiated detection performance across various combinations of source models and detection models. Under the $T = 0.2$ setting (Fig. 5(a)), the diagonal values generally show higher AUROC scores, indicating that the models perform better at detecting code generated by themselves. For example, CodeLlama demonstrates strong self-detection capability with an AUROC of 0.93; similarly, StarCoder achieves an exceptional self-detection AUROC of 0.96. In off-diagonal positions, which represent cross-model detection scenarios, the performance of DualCodeDetect only slightly declines, yet some detection models maintain stable performance across different source models. Among them, Qwen3-Coder shows particularly robust cross-model detection ability, maintaining AUROC values above 0.84 for most source models, suggesting that using a surrogate model is a feasible strategy in black-box detection scenarios. This trend also holds when GPT-5.2 and Claude-4.5 are treated as source models: despite the lack of internal logits, DualCodeDetect can still leverage open-source surrogate models to provide effective detection. However, code generated by CodeGen2 and Phi-1 proves significantly more challenging in cross-model detection, with most AUROC values below 0.80, likely due to the greater diversity in their training code corpora, which increases detection difficulty.

Under the $T = 1.0$ temperature setting (Fig. 5(b)), the detection performance of all model combinations noticeably declines, as increased generation randomness poses greater challenges to the detection task. Notably, at $T = 1.0$, code generated by CodeGen2 and CodeLlama as source models becomes particularly difficult to detect, with most AUROC values dropping below 0.70. The same degradation is also observed when closed-source models, GPT-5.2 and Claude-4.5, are used as source models, further indicating more diverse code generation patterns under high-temperature conditions. Nevertheless, using Qwen3-Coder as a surrogate model to detect code generated by GPT-5.2 still achieves an AUROC of 0.74.

In summary, under the black-box detection setting, DualCodeDetect enables effective cross-model detection via a surrogate model mechanism. The method demonstrates consistent robustness and practical utility across diverse model combinations, offering a reliable solution for identifying LLM-generated code in real-world applications, while the increased randomness under high-temperature settings can make black-box detection slightly more demanding in practice.

Answering RQ2: DualCodeDetect maintains reasonable performance in cross-model detection, demonstrating its effectiveness in black-box scenarios.

5.4 RQ3: Computation Cost

To evaluate the practical efficiency of DualCodeDetect, we compare its computational overhead with representative methods from the taxonomy established in §5.2 (RQ1), as follows: **(1) Supervised methods**, this category is excluded from direct comparison, since the training time involved is difficult to standardize and typically exceeds the inference time of zero-shot methods by orders of magnitude. **(2) Zero-shot statistical methods,** for which we choose Log Rank as a representative, incur the lowest overhead because they require only a single forward pass on the original code to compute logits. **(3) Perturbation-based zero-shot methods,** represented by DetectGPT and

Table 5. Performance of Different Perturbation Strategies. (Each cell reports AUROC↑ / FPR↓ / FNR↓)

Code LLM	Temperature	Perturbation Strategies			
		Random	Semantic	Structural	Semantic&Structural
Incoder (1.3B)	$T = 0.2$	0.8231 / 0.1666 / 0.1884	0.8611 / 0.1341 / 0.1507	0.9278 / 0.0525 / 0.0647	**0.9612 / 0.0313 / 0.0408**
	$T = 1.0$	0.6903 / 0.3052 / 0.3164	0.7467 / 0.2404 / 0.2588	0.7439 / 0.2429 / 0.2613	**0.7622 / 0.2155 / 0.2367**
WizardCoder (3B)	$T = 0.2$	0.8779 / 0.1124 / 0.1397	0.8748 / 0.1134 / 0.1452	0.9134 / 0.0549 / 0.0681	**0.9502 / 0.0377 / 0.0491**
	$T = 1.0$	0.7617 / 0.2181 / 0.2412	0.8325 / 0.1523 / 0.1733	0.8612 / 0.1109 / 0.1331	**0.8721 / 0.1085 / 0.1396**
CodeLlama (7B)	$T = 0.2$	0.5992 / 0.3712 / 0.4108	0.8375 / 0.1468 / 0.1712	0.8992 / 0.0823 / 0.1120	**0.9311 / 0.0459 / 0.0607**
	$T = 1.0$	0.6616 / 0.3147 / 0.3361	0.7034 / 0.2830 / 0.3016	0.7301 / 0.2579 / 0.2737	**0.7603 / 0.2105 / 0.2429**
DS-Coder-V2 (16B)	$T = 0.2$	0.7272 / 0.2569 / 0.2692	0.8326 / 0.1501 / 0.1643	0.8847 / 0.1036 / 0.1194	**0.9016 / 0.0571 / 0.0702**
	$T = 1.0$	0.7093 / 0.2608 / 0.2844	0.7316 / 0.2539 / 0.2810	0.7524 / 0.2263 / 0.2575	**0.7672 / 0.2209 / 0.2536**
Average	$T = 0.2$	0.7569 / 0.2268 / 0.2520	0.8515 / 0.1361 / 0.1579	0.9063 / 0.0733 / 0.0911	**0.9360 / 0.0430 / 0.0552**
	$T = 1.0$	0.7057 / 0.2747 / 0.2945	0.7536 / 0.2324 / 0.2537	0.7719 / 0.2095 / 0.2314	**0.7904 / 0.1889 / 0.2182**

DetectCodeGPT, whose runtime comprises three distinct components, i.e., logits calculation for the original code, perturbation generation, and logits calculation for the perturbed samples.

DualCodeDetect also adheres to the perturbation-based paradigm, with its workflow decomposed into four stages: (a) logits calculation for the original code, (b) semantic-channel perturbation, (c) structural-channel perturbation, and (d) logits calculation for the perturbed code.

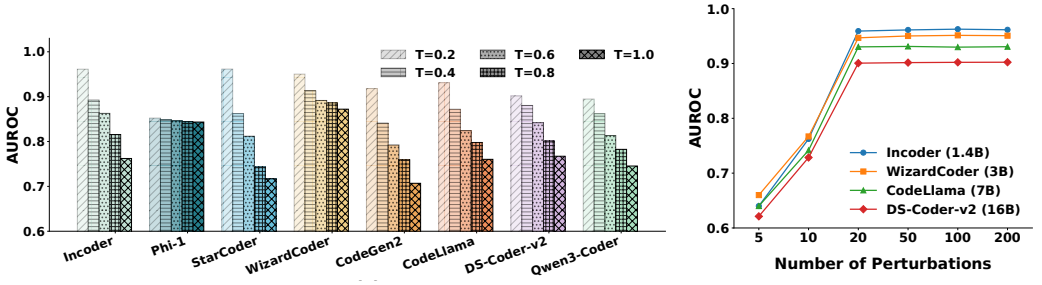
We uniformly employ CodeLlama as the detection model and record the total inference time across each stage of the detection pipeline for a same set of 500 samples on a single A6000 GPU. As shown in Table 4, the total inference time for DualCodeDetect is 3,264 seconds, the main computational bottleneck lies in LLM inference in stages (a) and (d), which together take 2,745 seconds, while perturbation stages (b) and (c) take 402 seconds and 117 seconds, respectively, indicating that the additional overhead introduced by dual-channel perturbation is relatively low.

Log Rank achieves the lowest runtime, as it requires only a single model inference, however as shown in §5.2, relying solely on raw probability statistics often fails to capture deeper features of LLM-generated code, resulting in substantially lower AUROC. Compared with DetectGPT, our DualCodeDetect requires only about 21% of the runtime. As analyzed in §5.5 (RQ4), DualCodeDetect converges with approximately 20 perturbations, whereas DetectGPT typically requires around 100, which greatly increases the number of expensive LLM inferences in stage (d). Notably, DetectCodeGPT also converges at about 20 perturbations and achieves a total runtime of 2,754 seconds, slightly faster than DualCodeDetect due to its lightweight whitespace-based perturbations, however our DualCodeDetect yields higher detection accuracy at the cost of only a modest increase in runtime, resulting in a more favorable efficiency–accuracy trade-off for practical deployment.

Answering RQ3: DualCodeDetect achieves reasonable computational efficiency while maintaining high detection performance, significantly reducing the time overhead to approximately 21% of that required by DetectGPT.

5.5 RQ4: Ablation Study

This section evaluates the effectiveness of different perturbation strategies in detecting LLM-generated code through an ablation study. Under limited computational resources, we select four representative models of varying parameter scales, namely Incoder (1.3B), WizardCoder (3B), CodeLlama (7B), and DeepSeek-Coder-V2 (16B), and conduct experiments on the CodeSearchNet dataset. Four perturbation configurations are compared: 1) random perturbation (i.e., NPR-enhanced DetectGPT), 2) semantic channel perturbation only, 3) structural channel perturbation only, and 4) dual-channel perturbation combining both semantic and structural perturbations (i.e., DualCodeDetect), in order to comprehensively assess the contribution of each component.



(a) Impact of the temperature of LLMs (b) Impact of the number of perturbations

Fig. 6. The Impact of Parameter Settings on Detection Performance.

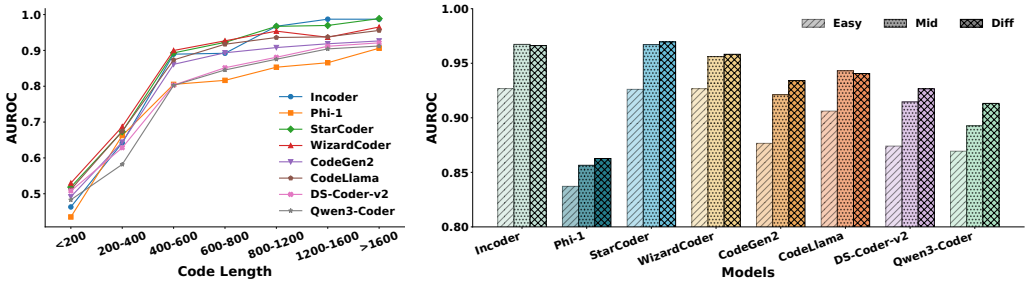
As shown in Table 5, the proposed dual-channel perturbation strategy significantly outperforms the random perturbation baseline across different generative models and temperature settings. At $T = 0.2$, compared to the random perturbation, the semantic-only and structural-only perturbations achieve average AUROC scores of 0.8515 and 0.9036, respectively, representing performance improvements of 12.5% and 19.7%. This demonstrates that both perturbation strategies effectively enhance the discriminative power against LLM-generated code. Furthermore, when semantic and structural channels operate synergistically, the detection performance reaches an average AUROC of 0.9360 while maintaining remarkably low FPR and FNR of 0.0430 and 0.0552, respectively. Under higher randomness conditions ($T = 1.0$), all perturbation methods exhibit performance degradation, yet DualCodeDetect maintains superior detection capability (average AUROC is 0.7904), outperforming both random perturbation and single-channel configurations. This reveals synergistic effects between the dual-channel perturbation strategies.

Answering RQ4: The collaborative perturbation of the semantic and structural channels consistently shows superior performance under both temperatures, confirming the significant effectiveness of the dual-channel perturbation strategy for LLM-generated code detection.

5.6 RQ5: Impact of Parameter Settings

To investigate the impact of different parameter settings on the detection performance of DualCodeDetect, we conduct experiments on the CodeSearchNet dataset under varying LLM temperature parameters and different numbers of perturbations. Notably, given space constraints, we report only AUROC, and using FPR or FNR leads to the same conclusions. The results are shown in Figure 6.

Sampling temperature controls the randomness of LLM generation by adjusting the smoothness of the probability distribution, thereby influencing the diversity and determinism of the generated outputs. To analyze the effect of temperature on the detection performance of DualCodeDetect, we evaluate the code generated by eight open-source different code LLMs at temperatures $T = \{0.2, 0.4, 0.6, 0.8, 1.0\}$. As shown in Figure 6(a), at $T = 0.2$, DualCodeDetect achieves the best detection performance across all eight code LLMs. As the temperature increases, the diversity of the generated code also increases, leading to a noticeable decline in detection performance. We attribute this drop to the fact that increasing temperature flattens the softmax output distribution, making token sampling less concentrated and more stochastic, which makes the statistical characteristics of LLM-generated code move closer to the "natural randomness" observed in human-written code. DualCodeDetect relies on actively introducing perturbations to amplify the distributional gap between original and perturbed samples. However, under high-temperature generation, the sampling process itself already injects a certain level of "spontaneous perturbation," which partially overlaps with our designed perturbations. As a result, the relative divergence between the original code and its perturbed variants decreases, weakening the separability of NPR-based scores.



(a) Impact of Code Length (b) Impact of Code Complexity

Fig. 7. The Impact of Code Attributes on Detection Performance (AUROC).

Table 6. The Impact of Code Correctness on Detection Performance (AUROC).

Type	Code LLM								Average
	Incoder	Phi-1	StarCoder	WizardCoder	CodeGen2	CodeLlama	DS-Coder-V2	Qwen3-Coder	
Correct	0.9641	0.8536	0.9630	0.9508	0.9194	0.9315	0.9055	0.8974	**0.9232**
Incorrect	0.8834	0.8121	0.8745	0.8719	0.8466	0.8512	0.8461	0.8405	0.8533

The number of perturbations is another key parameter influencing the detection performance. Under limited experimental resources, consistent with §5.5, we select four representative models of different parameter scales, including Incoder (1.3B), WizardCoder (3B), CodeLlama (7B), and DeepSeek-Coder-V2 (16B), and further investigate the impact of the number of perturbations on detection performance at a temperature setting of $T = 0.2$. The results in Figure 6(b) show that as the number of perturbations increases, the AUROC score gradually improves and eventually stabilizes. When the number of perturbations reaches 20, the method already achieves robust detection performance; further increasing the number of perturbations provides limited improvement in performance while significantly increasing computational overhead. These results demonstrate that DualCodeDetect can effectively distinguish between LLM-generated and human-written code with a relatively small number of perturbations, indicating high computational efficiency.

Answering RQ5: DualCodeDetect shows superior performance in detecting generated code under low-temperature settings, though its effectiveness decreases to some extent as the temperature parameter increases. Moreover, the method achieves robust detection performance with a relatively small number of perturbations, ensuring high computational efficiency.

5.7 RQ6: Impact of Code Attributes

To investigate the impact of code attributes on the detection performance of DualCodeDetect, we conduct experiments on the CodeSearchNet dataset using eight open-source LLMs, focusing on three aspects: code length, code complexity, and code correctness. And we also only report AUROC.

Regarding code length, we categorize code samples into different intervals based on the number of characters and evaluate the detection performance within each interval. As shown in Figure 7(a), AUROC generally increases with longer code. When the code length exceeds 800 characters, the detection performance gradually stabilizes and remains at a high level (AUROC ≥ 0.9). This result indicates that longer code typically contains richer structural and semantic information, helping DualCodeDetect more accurately distinguish between LLM-generated and human-written code.

For code complexity, we leverage the Halstead complexity [13] measure to divide the code samples evenly into three levels: “easy”, “medium”, and “difficult”. According to the results in Figure 7(b), DualCodeDetect maintains strong detection performance across all complexity levels (AUROC $\geq$

0.85), with particularly superior results on highly complex code. Code with higher complexity often exhibits more diverse structural patterns and logical combinations, providing greater transformation opportunities for the dual-channel perturbation strategy and thereby enhancing DualCodeDetect's ability to capture differences between generated and human-written code.

Additionally, we analyze the impact of code correctness on detection performance. As shown in Table 6, for syntactically correct code, DualCodeDetect achieves an average AUROC of 0.9232 across all models, slightly higher than the 0.8533 observed for incorrect code. This result underscores the importance of code correctness in detection effectiveness. Potential reasons include two aspects: first, correct and incorrect code inherently differ in syntactic and structural distributions; second, syntax errors, as we discussed in §4.3.2, may reduce the number of applicable code transformation rules in the structural channel, thereby weakening the structural perturbation strength. However, thanks to our syntax-error-robust design, the structural channel does not collapse on incorrect code and can still operate on the well-formed regions that remain parseable.

Answering RQ6: The detection performance of DualCodeDetect is systematically influenced by multiple code attributes. Specifically, detection efficacy improves markedly with increasing code length, as longer sequences provide more discriminative features. Code complexity is also positively correlated with detection performance, where structurally complex code enhances the method's ability to distinguish between sources. Furthermore, compared to syntactically incorrect code, syntactically correct code exhibits a modest advantage in detection accuracy.

6 Discussion

6.1 Generalizability to Other Programming Languages

The DualCodeDetect framework is language-agnostic. To verify its scalability, we extend our evaluation to include Java, C++, and JavaScript.

The semantic channel perturbation can be directly applied to other languages without modification. For the structural channel, 11 of the original 14 rules are generic across most languages, requiring only minor syntactic adjustments. We exclude the Python-specific rules (2), (3) and (4) and introduce language-specific equivalent transformations:

- Java: Explicit "`this`" (`field` ↔ `this.field`) and array declaration (`int[] a` ↔ `int a[]`).
- C++: Supplement rule (7) (`i++` and `++i`) and IO stream style (`std::cout` ↔ `printf`).
- JavaScript: Template literals (`"s"+v` ↔ `'s$v'`) and property access notation (`obj.p` ↔ `obj["p"]`).

We utilize the Java and JavaScript datasets from CodeSearchNet, and the C++ dataset from The Stack, as CodeSearchNet does not include C++. Using four code LLMs of varying scales at a temperature setting of $T = 0.2$, we compare DualCodeDetect against three strong baseline methods (Log Rank, DetectGPT, and DetectCodeGPT). As shown in Table 7, DualCodeDetect consistently achieves superior performance across all languages, confirming its strong cross-language scalability.

6.2 Lessons Learned

Perturbation Strategy. The dual-channel perturbation strategy effectively expands the perturbation space in low-entropy structured data by collaboratively applying perturbations at both semantic and structural levels. In contrast to random perturbations, which often introduce subtle errors that distort statistical score calculations, our structural channel incorporates semantically equivalent transformation rules derived from code style entropy analysis. This approach systematically introduces structural diversity while preserving code correctness, thereby ensuring the reliability of the detection method. The proposed strategy offers an effective and robust perturbation framework for detecting generated content in low-entropy domains.

Zero-Shot Detection. With the increasing diversity and rapid iteration of code generation models, supervised learning-based detection methods face significant generalization challenges. Such

Table 7. Performance of Different Programming Languages. (Each cell reports AUROC↑ / FPR↓ / FNR↓)

Language	Code LLM	Detection Methods			
		Log Rank	DetectGPT	DetectCodeGPT	**DualCodeDetect**
Java	Incoder (1.3B)	0.9223 / 0.0478 / 0.0641	0.6271 / 0.3538 / 0.3716	0.9362 / 0.0464 / 0.0582	**0.9381 / 0.0433 / 0.0572**
	WizardCoder (3B)	0.8817 / 0.1034 / 0.1197	0.6733 / 0.3196 / 0.3354	0.9085 / 0.0733 / 0.0884	**0.9129 / 0.0528 / 0.0693**
	CodeLlama (7B)	0.8844 / 0.0989 / 0.1102	0.7069 / 0.2848 / 0.3125	0.8854 / 0.1036 / 0.1159	**0.9046 / 0.0578 / 0.0722**
	DS-Coder-V2 (16B)	0.8419 / 0.1317 / 0.1580	0.7231 / 0.2605 / 0.2859	0.8739 / 0.1094 / 0.1277	**0.8914 / 0.0864 / 0.1096**
C++	Incoder (1.3B)	0.9280 / 0.0521 / 0.0633	0.6389 / 0.3442 / 0.3628	**0.9344 / 0.0475** / 0.0600	0.9302 / 0.0506 / **0.0549**
	WizardCoder (3B)	0.8901 / 0.0932 / 0.1118	0.6512 / 0.3215 / 0.3316	0.8996 / 0.0752 / 0.0985	**0.9095 / 0.0711 / 0.0892**
	CodeLlama (7B)	0.8655 / 0.1328 / 0.1495	0.7414 / 0.2466 / 0.2703	0.8752 / 0.1120 / 0.1309	**0.8917 / 0.0821 / 0.0994**
	DS-Coder-V2 (16B)	0.8223 / 0.1704 / 0.1865	0.7251 / 0.2580 / 0.2735	0.8649 / 0.1311 / 0.1481	**0.8991 / 0.0816 / 0.1044**
JavaScript	Incoder (1.3B)	0.8784 / 0.1106 / 0.1463	0.6032 / 0.3685 / 0.4072	0.9044 / 0.0566 / 0.0718	**0.9190 / 0.0539 / 0.0667**
	WizardCoder (3B)	0.8627 / 0.1315 / 0.1491	0.5881 / 0.3903 / 0.4255	0.8827 / 0.1064 / 0.1203	**0.9102 / 0.0562 / 0.0714**
	CodeLlama (7B)	0.8602 / 0.1362 / 0.1515	0.5953 / 0.3734 / 0.4141	0.8796 / 0.1043 / 0.1381	**0.8839 / 0.1039 / 0.1200**
	DS-Coder-V2 (16B)	0.8315 / 0.1558 / 0.1810	0.5919 / 0.3780 / 0.4196	0.8731 / 0.1074 / 0.1399	**0.8792 / 0.1102 / 0.1417**
Average		0.8724 / 0.1137 / 0.1326	0.6555 / 0.3249 / 0.3508	0.8932 / 0.0894 / 0.1082	**0.9058 / 0.0708 / 0.0880**

approaches rely on large amounts of labeled data to train classifiers, requiring not only coverage of existing generative models but also frequent updates to adapt to emerging ones, which is a process that is difficult to sustain in practice. In contrast, the zero-shot detection paradigm adopted in this work does not depend on any training data; instead, it leverages the inherent statistical characteristics of generative models themselves for discrimination, thereby circumventing issues of model coverage and the need for continuous retraining. Thus, zero-shot detection is particularly suitable for real-world scenarios involving diverse and origin-unknown code detection tasks, offering a feasible pathway to address challenges posed by the rapid evolution of generative models.

6.3 Limitations

Robustness under High Temperature. The detection performance of DualCodeDetect decreases as the sampling temperature increases. However, as discussed in §5.6, higher temperatures reduce the statistical separability between LLM-generated and human-written code, a common challenge faced by all zero-shot detectors that rely on probability-based statistical signals. Nevertheless, under equally difficult settings, DualCodeDetect consistently demonstrates superior robustness compared to baseline methods. This resilience is attributed to our dual-channel complementary design, which provides stronger discriminative signals than detectors relying solely on semantic features.

Detection of Incorrect Code. DualCodeDetect experiences a slight degradation on syntactically incorrect code, as syntax errors disrupt structural transformation rules and limit the perturbation strategy. However, relying on local pattern matching, the structure-aware perturbation bypasses erroneous regions to transform valid segments. Thus, the structural channel is not rendered entirely ineffective; only the applicable rules are reduced. Future work will explore perturbation strategies less dependent on syntactic correctness to further boost robustness and generalizability.

7 Conclusion

In this study, we propose DualCodeDetect, a zero-shot detection framework based on a dual-channel perturbation mechanism for identifying code generated by LLMs. The method employs an out-of-nucleus sampling-based identifier perturbation strategy in the semantic channel and incorporates semantically equivalent structural transformation rules in the structural channel. These two channels work synergistically to effectively address the challenge of limited perturbation space in low-entropy programming languages, while preserving semantic integrity and syntactic correctness during perturbation. Comprehensive experiments on two real-world datasets and ten mainstream code generation LLMs, including eight open-source and two closed-source models, show that DualCodeDetect significantly outperforms all existing baselines in detection performance.

Data Availability

To facilitate future research, the core implementation of DualCodeDetect, including the dual-channel perturbation mechanisms and the key detection algorithms, is publicly available on GitHub at https://github.com/Doug17i/DualCodeDetect.

Acknowledgments

This work was supported in part by National Natural Science Foundation of China (Grants U2436601 and 62472398) and by the New Generation Artificial Intelligence-National Science and Technology Major Project (No. 2025ZD0123202).

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Anthropic. 2025. Claude 4.5 Sonnet. https://claude.ai/. Accessed: 2025-12-24.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[4] Guangsheng Bao, Yanbin Zhao, Zhiyang Teng, Linyi Yang, and Yue Zhang. 2024. Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature. In *International Conference on Learning Representations*, Vol. 2024. 24814–24836.

[5] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023).

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Paul Denny, James Prather, Brett A Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing education in the era of generative AI. *Commun. ACM* 67, 2 (2024), 56–67. doi:10.1145/3624720

[8] Damian Okaibedi Eke. 2023. ChatGPT and the rise of generative AI: Threat to academic integrity? *Journal of Responsible Technology* 13 (2023), 100060. doi:10.1016/j.jrt.2023.100060

[9] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).

[10] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M Rush. 2019. Gltr: Statistical detection and visualization of generated text. In *Proceedings of the 57th annual meeting of the association for computational linguistics: system demonstrations*. 111–116. doi:10.18653/v1/p19-3019

[11] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. 2023. Textbooks are all you need. *arXiv preprint arXiv:2306.11644* (2023).

[12] Farrokh Habibzadeh. 2023. GPTZero performance in identifying artificial intelligence-generated medical texts: a preliminary study. *Journal of Korean medical science* 38, 38 (2023). doi:10.3346/jkms.2023.38.e319

[13] T Hariprasad, G Vidhyagaran, K Seenu, and Chandrasegar Thirumalai. 2017. Software complexity analysis using halstead metrics. In *2017 international conference on trends in electronics and informatics (ICEI)*. IEEE, 1109–1113. doi:10.1109/icoei.2017.8300883

[14] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131. doi:10.1145/2442754.2442763

[15] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751* (2019).

[16] Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. 2023. Radar: Robust ai-text detection via adversarial learning. *Advances in neural information processing systems* 36 (2023), 15077–15095. doi:10.52202/075280-0662

[17] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

[18] Daphne Ippolito, Daniel Duckworth, Chris Callison-Burch, and Douglas Eck. 2020. Automatic detection of generated text is easiest when humans are fooled. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 1808–1822. doi:10.18653/v1/2020.acl-main.164

[19] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. 2023. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli calling international conference on computing education research*. 1–12. doi:10.1145/3631802.3631806

[20] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. 2022. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533* (2022).

[21] Taehyun Lee, Seokhee Hong, Jaewoo Ahn, Ilgee Hong, Hwaran Lee, Sangdoo Yun, Jamin Shin, and Gunhee Kim. 2024. Who wrote this code? watermarking for code generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4890–4911. doi:10.18653/v1/2024.acl-long.268

[22] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[23] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158

[24] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[25] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. Wizardcoder: Empowering code large language models with evol-instruct. In *International Conference on Learning Representations*, Vol. 2024. 27168–27188.

[26] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. 2023. Detectgpt: Zero-shot machine-generated text detection using probability curvature. In *International conference on machine learning*. PMLR, 24950–24962.

[27] Phuong T Nguyen, Juri Di Rocco, Claudio Di Sipio, Riccardo Rubei, Davide Di Ruscio, and Massimiliano Di Penta. 2024. GPTSniffer: A CodeBERT-based classifier to detect source code written by ChatGPT. *Journal of Systems and Software* 214 (2024), 112059. doi:10.1016/j.jss.2024.112059

[28] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. Codegen2: Lessons for training llms on programming and natural languages. *arXiv preprint arXiv:2305.02309* (2023).

[29] Wei Hung Pan, Ming Jie Chok, Jonathan Leong Shan Wong, Yung Xin Shin, Yeong Shian Poon, Zhou Yang, Chun Yong Chong, David Lo, and Mei Kuan Lim. 2024. Assessing ai detectors in identifying ai-generated code: Implications for education. In *Proceedings of the 46th international conference on software engineering: software engineering education and training*. 1–11. doi:10.1145/3639474.3640068

[30] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105. doi:10.1109/sp46214.2022.9833571

[31] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[32] Sapling Intelligence. 2025. Sapling AI Content Detector. https://sapling.ai/ai-content-detector. Accessed: 2025-06-01.

[33] Claude E Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal* 27, 3 (1948), 379–423.

[34] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. 2025. Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1628–1639. doi:10.1109/icse55347.2025.00005

[35] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267* (2025).

[36] Irene Solaiman, Miles Brundage, Jack Clark, Amanda Askell, Ariel Herbert-Voss, Jeff Wu, Alec Radford, Gretchen Krueger, Jong Wook Kim, Sarah Kreps, et al. 2019. Release strategies and the social impacts of language models. *arXiv preprint arXiv:1908.09203* (2019).

[37] Jinyan Su, Terry Zhuo, Di Wang, and Preslav Nakov. 2023. Detectllm: Leveraging log rank information for zero-shot detection of machine-generated text. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 12395–12412. doi:10.18653/v1/2023.findings-emnlp.827

[38] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388
[39] Chip Thien. [n. d.]. gpt-2-output-dataset. https://github.com/MacroChip/gpt-2-output-dataset. Accessed: 2025-06-01.
[40] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 conference on empirical methods in natural language processing*. 8696–8708. doi:10.18653/v1/2021.emnlp-main.685
[41] Michel Wermelinger. 2023. Using github copilot to solve simple programming problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 172–178. doi:10.1145/3545945.3569830
[42] Zhenyu Xu and Victor S Sheng. 2024. Detecting AI-generated code assignments using perplexity of large language models. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 38. 23155–23162. doi:10.1609/aaai.v38i21.30361
[43] Xianjun Yang, Wei Cheng, Yue Wu, Linda Petzold, William Wang, and Haifeng Chen. 2024. Dna-gpt: Divergent n-gram analysis for training-free detection of gpt-generated text. In *International Conference on Learning Representations*, Vol. 2024. 48572–48597.
[44] Xianjun Yang, Kexun Zhang, Haifeng Chen, Linda Petzold, William Yang Wang, and Wei Cheng. 2023. Zero-shot detection of machine-generated codes. *arXiv preprint arXiv:2310.05103* (2023).
[45] Tong Ye, Yangkai Du, Tengfei Ma, Lingfei Wu, Xuhong Zhang, Shouling Ji, and Wenhai Wang. 2025. Uncovering llm-generated code: A zero-shot synthetic code detector via code rewriting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 968–976. doi:10.1609/aaai.v39i1.32082
[46] Rowan Zellers, Ari Holtzman, Hannah Rashkin, Yonatan Bisk, Ali Farhadi, Franziska Roesner, and Yejin Choi. 2019. Defending against neural fake news. *Advances in neural information processing systems* 32 (2019). doi:10.1108/sd-07-2019-0139
[47] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5673–5684. doi:10.1145/3580305.3599790
[48] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024).

Received 2026-02-24; accepted 2026-03-24